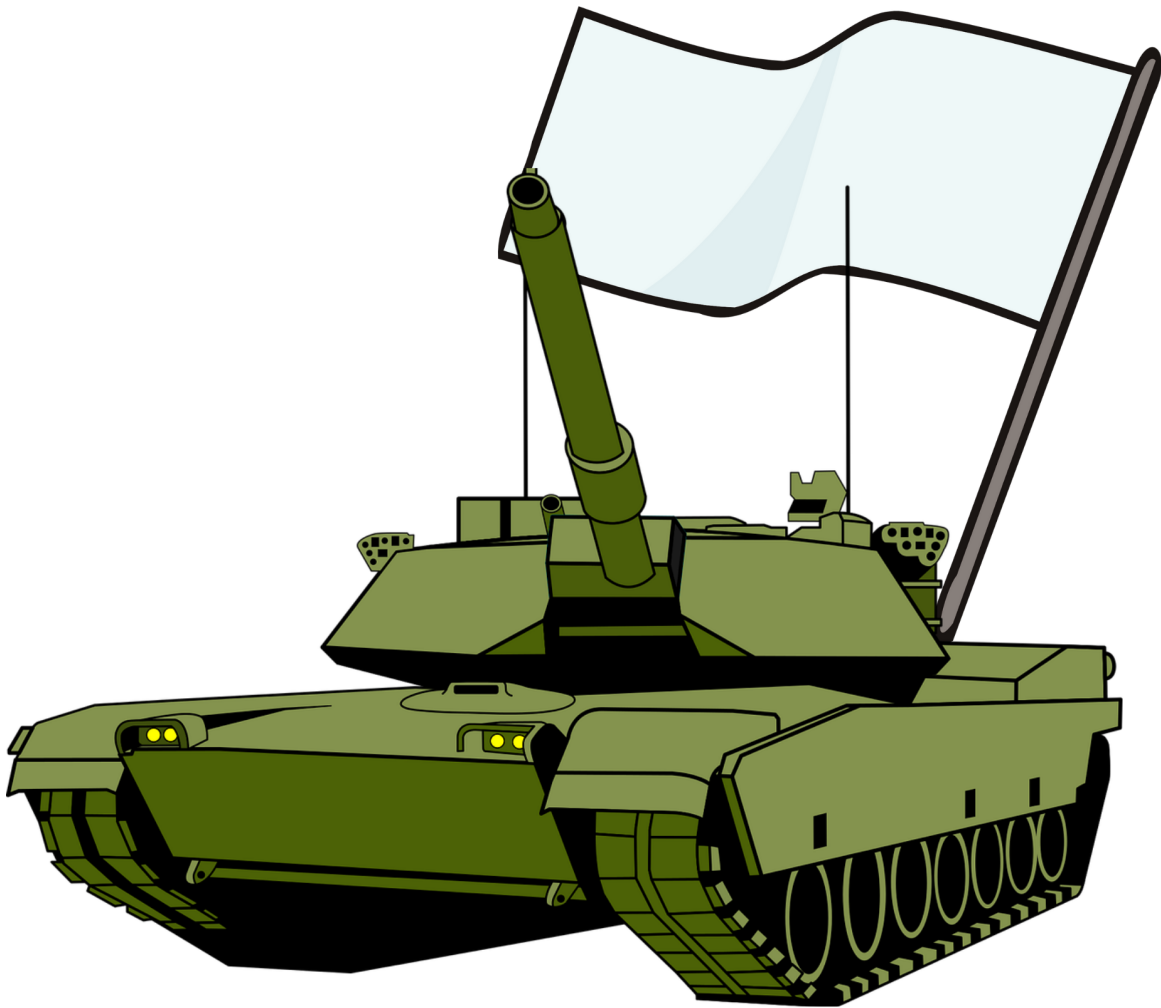


USER MANUAL



CAPTURE THE FLAG

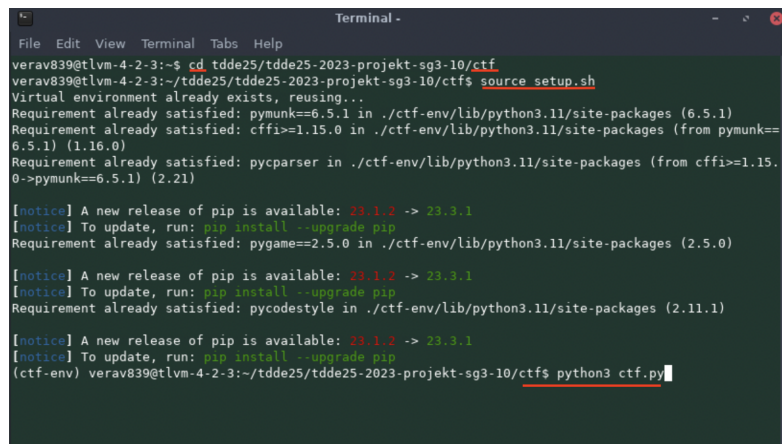


Starting the Game

Make sure you have python3 installed on your computer and when that's done, get the game files from the provided git repository.

Now it's time to open the terminal and go to the game directory, then run the following commands to start the game:

- `source setup.sh`
- `python3 ctf.py`



```
Terminal -
File Edit View Terminal Tabs Help
verav839@tlvm-4-2-3:~$ cd tdde25/tdde25-2023-projekt-sg3-10/ctf
verav839@tlvm-4-2-3:~/tdde25/tdde25-2023-projekt-sg3-10/ctf$ source setup.sh
Virtual environment already exists, reusing...
Requirement already satisfied: pymunk==6.5.1 in ./ctf-env/lib/python3.11/site-packages (6.5.1)
Requirement already satisfied: cffi>=1.15.0 in ./ctf-env/lib/python3.11/site-packages (from pymunk==6.5.1) (1.16.0)
Requirement already satisfied: pycparser in ./ctf-env/lib/python3.11/site-packages (from cffi>=1.15.0->pymunk==6.5.1) (2.21)

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: pip install --upgrade pip
Requirement already satisfied: pygame==2.5.0 in ./ctf-env/lib/python3.11/site-packages (2.5.0)

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: pip install --upgrade pip
Requirement already satisfied: pycodestyle in ./ctf-env/lib/python3.11/site-packages (2.11.1)

[notice] A new release of pip is available: 23.1.2 -> 23.3.1
[notice] To update, run: pip install --upgrade pip
(ctf-env) verav839@tlvm-4-2-3:~/tdde25/tdde25-2023-projekt-sg3-10/ctf$ python3 ctf.py
```

File Overview

In the source folder you will find the following files:

- **ctf.py:** The main file of the game where the main program runs the game loop. Imports necessary libraries and modules, and handles initialization, game setup, collision handling and game logic. It also utilizes functions for creating game elements and updating their states. It incorporates AI decisions, explosions- and sound effects, and multiplayer functionality.
- **gameobjects.py:** This file contains classes and functions to model and manage various game elements/objects in the game environment. These objects include tanks, boxes, bullets, flags and explosions. It covers both visual representation and physical interactions.
- **ai.py:** Contains classes and functions related to artificial intelligence (AI) used in the game. The AI is designed to control tanks, make decisions, and perform acting such as shooting, moving and turning.
- **images.py:** This file serves as a central location for managing and accessing graphical assets used in the game. Images are loaded and stored for various game elements/objects, facilitating their use in rendering the game environment.
- **maps.py:** The provided code in this file, defines a "Map" class and contains a "map_list" with instances of this class by reading map information from a JSON file in a specified directory "maps".

Features Overview

These are the implemented features:

- **Recoil:** When a tank fires a shot, it experiences acceleration in the opposite direction of the bullet's trajectory. This function is integrated into the `gameobjects.py` file within the bullet class.
- **Unfair AI:** This feature increases the difficulty level against AI opponents by providing them with a permanent buff, which is a boolean given from the "Unfair-AI"- variable in `ctf.py` file. If you want to turn this feature off, you have to set the "Unfair-AI" to false.
- **Sounds:** We've incorporated shooting sounds, collision effects with game objects, and a distinctive sound for flag pickup. The shooting and flag pickup sounds are specifically located in the `gameobjects.py` file, while the remaining sounds are housed in `ctf.py`. Additionally, we've added engaging background music.
- **Explosions:** This feature introduces an explosion sprite that appears when a bullet collides with either a tank or a destructible box. The implementation involves an 'Explosion' class located in the `gameobjects.py` file, complemented by a corresponding 'remove_explosion' function within `ctf.py`.
- **JSON File:** Instead of storing map data in a Python file, we store it in a JSON file. This modification enables the game to read map data from the JSON file which improves the modifiability, ease of map creation, and compatibility with external sources.

There are three maps to choose from; "map0", "map1" and "map2". To choose a map, you have to write which map you want in the terminal when starting the game. It should look like this: `python3 ctf.py -map2`. If you don't write anything when starting the game, then "map0" will be chosen.

- **Hit Points:** This feature makes it harder to destroy tanks and destructible boxes. These hit points are managed in the collision handlers (in `ctf.py`).
- **Counting Score:** The game includes a scoring feature that tallies and displays player scores on the console output when a player successfully retrieves the flag and returns it to the base. This functionality is invoked through the 'score_counter' function located in `ctf.py`.
- **Hot Seat Multiplayer:** Makes it possible for you to play with another person on the same computer. To activate this feature, include the "--hot-multiplayer" command when starting the game. For instance, the command would look like: `python3 ctf.py --hot-multiplayer`. If you don't write this, the game will automatically be "singleplayer". To combine multiplayer with a specific map, use a command like: `python3 ctf.py -map2 --hot-multiplayer`.

In multiplayer mode, one player controls movement with the WASD keys and shoots using the spacebar, while the other player uses the arrow keys for movement and the 'enter' key to shoot. In singleplayer mode, movement is controlled with the arrow keys, and shooting is done with the space bar.